

CLOUD COMPUTING

Cloud Management & Software Deployment

Resource provisioning • Load balancing & auto-scaling • Cloud orchestration with Kubernetes • Resilience, flexibility & cost-efficiency

Learning Objectives

- Explain the meaning of cloud management.
- Describe how cloud resources are provisioned.
- Understand the purpose of load balancing.
- Explain how auto-scaling improves application performance.
- Identify the role of Kubernetes in cloud orchestration.
- Discuss ways to design cloud systems for resilience and cost-efficiency.

What Is Cloud Management?

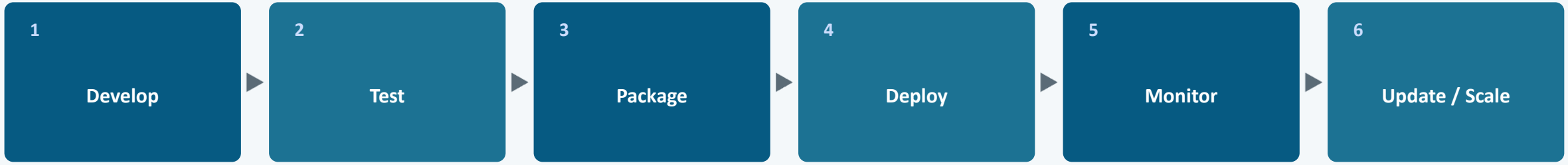
Cloud management is the process of controlling and monitoring cloud-based:

- **Computing resources**
- **Storage**
- **Networks**
- **Applications**
- **Security policies**
- **Users & access permissions**

Cloud management helps organizations ensure resources are available, secure, efficiently used, properly monitored, and cost-effective.

What Is Software Deployment?

Software deployment is the process of making an application available for users.



Cloud platforms simplify deployment

Cloud platforms simplify deployment by providing on-demand infrastructure and automated tools — removing much of the manual effort involved in packaging, releasing, and monitoring an application across environments.

Cloud Service Models

Model	Description	Example
Infrastructure as a Service (IaaS)	Provides virtual machines, networks, and storage	Amazon EC2
Platform as a Service (PaaS)	Provides a platform for developing and deploying applications	Google App Engine
Software as a Service (SaaS)	Provides ready-to-use software over the internet	Microsoft 365
Containers as a Service (CaaS)	Provides tools for running and managing containers	Amazon ECS

Cloud management can be applied across all of these service models.

Resource Provisioning

Resource provisioning is the process of allocating computing resources to an application — virtual machines, containers, databases, storage, network connections, and security groups.

Traditional Provisioning

- Often performed manually
- Can be slow
- May result in over-provisioning
- Difficult to reproduce consistently

Cloud Provisioning

- Can be automated
- Uses templates and scripts
- Supports rapid deployment
- Can respond to changing demand

Infrastructure as Code (IaC)

IaC means managing infrastructure through code instead of manual configuration.

1. **Write configuration**
2. **Review code**
3. **Deploy infrastructure**
4. **Monitor resources**

COMMON IaC TOOLS

- Terraform
- AWS CloudFormation
- Azure Resource Manager
- Ansible

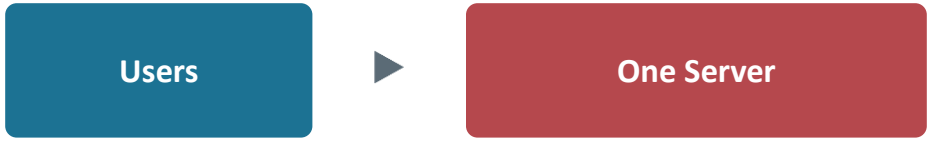
BENEFITS

- Repeatable deployments
- Fewer configuration errors
- Faster infrastructure creation
- Easier version control
- Improved team collaboration

Load Balancing

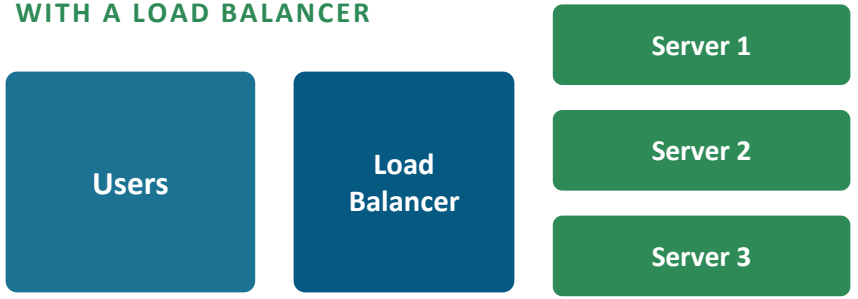
A load balancer distributes incoming network traffic among multiple servers or application instances.

WITHOUT A LOAD BALANCER



If the server becomes overloaded, the application may become slow or unavailable.

WITH A LOAD BALANCER



Improved performance

Higher availability

Better use of resources

Reduced risk of failure

Supports horizontal scaling

Types of Load Balancing

Application Load Balancing

Works at the application layer; routes traffic based on URL paths, hostnames, HTTP headers, and application content.

Network Load Balancing

Works at the network/transport layer. Useful for high-performance applications, TCP/UDP traffic, and low-latency communication.

Global Load Balancing

Distributes users across multiple geographic regions — reducing network latency, regional failures, and pressure on one data center.

Auto-Scaling

Auto-scaling automatically increases or decreases cloud resources according to demand.

2	8	2
Normal traffic application instances	High traffic application instances	Low traffic application instances

COMMON SCALING METRICS

- **CPU utilization**
- **Memory usage**
- **Number of requests**
- **Queue length**
- **Response time**
- **Schedule / time of day**

Horizontal and Vertical Scaling

Scaling Type	Meaning	Example
Horizontal scaling	Adding or removing instances	Increasing servers from 2 to 6
Vertical scaling	Increasing the power of an existing server	Adding more CPU or memory
Automatic scaling	Resources change automatically based on rules	Adding servers when CPU exceeds 70%

Horizontal scaling is often preferred in cloud environments because it improves availability and supports distributed applications.

Benefits & Challenges of Auto-Scaling

Benefits

- Handles unexpected traffic
- Improves application availability
- Reduces manual work
- Avoids paying for unnecessary resources
- Supports business growth

Challenges

- Poorly configured rules may cause instability
- Scaling may take time
- Some applications are difficult to scale
- Databases may become bottlenecks
- Sudden traffic increases can raise costs

Auto-scaling should be combined with monitoring, testing, and cost controls.

Introduction to Kubernetes

Kubernetes is an open-source platform for deploying, managing, and scaling containerized applications. It helps organizations:

- **Run containers across multiple machines**
- **Automatically restart failed containers**
- **Scale applications**
- **Perform software updates**
- **Manage networking and service discovery**
- **Maintain the desired application state**

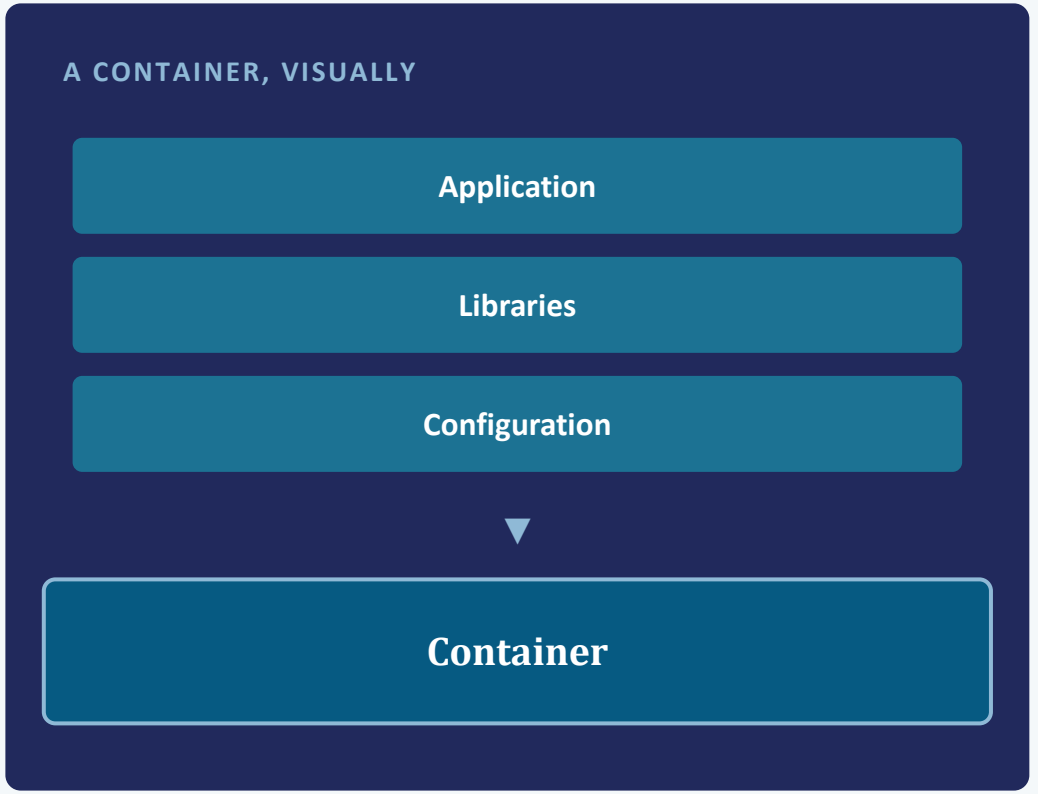
Kubernetes is often called a container orchestration platform.

Why Use Containers?

A container packages an application together with its dependencies.

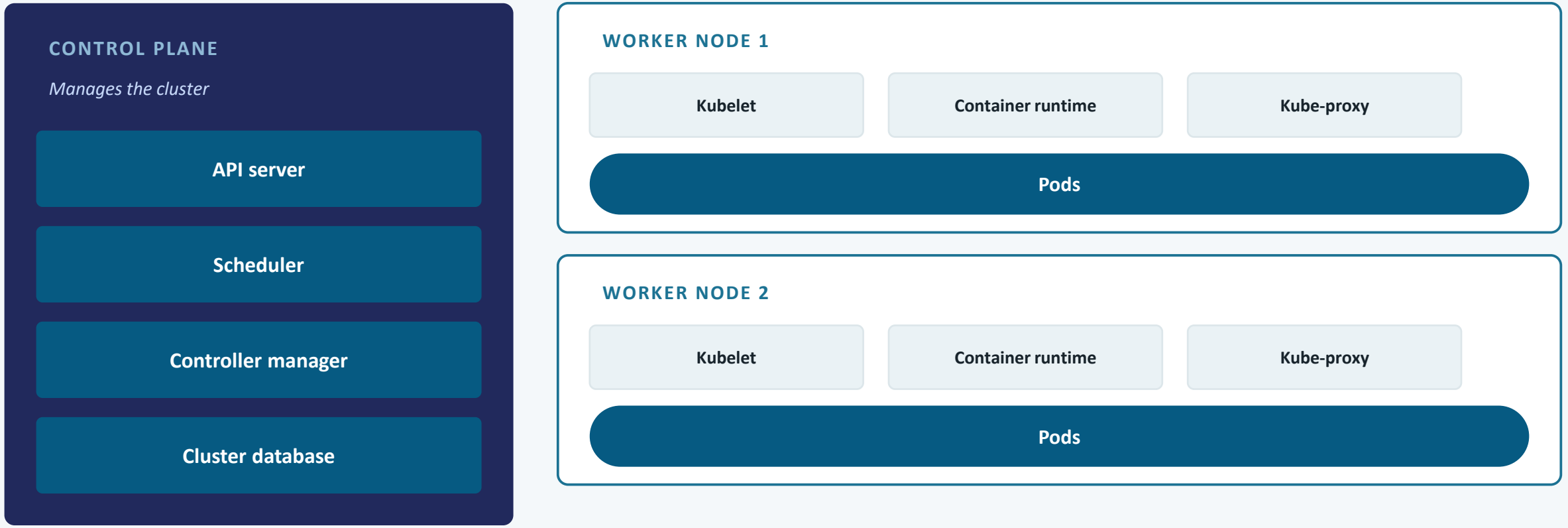
ADVANTAGES

- Consistent behavior across environments
- Faster startup than virtual machines
- Efficient use of resources
- Easier application deployment
- Simplified dependency management



Kubernetes Architecture

A Kubernetes cluster contains a Control Plane and one or more Worker Nodes.





Important Kubernetes Objects



Pod

The smallest deployable unit in Kubernetes. Contains one or more containers.



Deployment

Describes the desired number and version of application instances.



Service

Provides stable network access to a group of Pods.



Ingress

Controls external HTTP and HTTPS access to applications.



ConfigMap & Secret

Store configuration values and sensitive information.



Namespace

Separates resources within a cluster.

Kubernetes Deployment Process



A TYPICAL DEPLOYMENT PROCESS

- Build the application.
- Create a container image.
- Store the image in a container registry.
- Write a Kubernetes deployment file.
- Apply the configuration to the cluster.
- Kubernetes creates the required Pods.
- A Service exposes the application.
- Monitoring tools track performance and health.

Kubernetes Self-Healing & Updates

Self-Healing

- Restarting failed containers
- Replacing unhealthy Pods
- Rescheduling workloads on available nodes
- Maintaining the desired number of replicas

Also Supports

- Rolling updates
- Rollbacks
- Health checks
- Automated service discovery

This reduces downtime and simplifies software deployment.

Designing for Resilience

Resilience is the ability of a system to continue operating when failures occur.

- Use multiple servers or availability zones.
- Avoid single points of failure.
- Use load balancers.
- Replicate important data.
- Automate recovery procedures.
- Use health checks.
- Back up data regularly.
- Test disaster recovery plans.

A resilient system expects that failures will occur — and prepares for them.

Designing for Flexibility

Flexible cloud systems can adapt to changing business and technical requirements.

- Use modular application components.
- Separate services where appropriate.
- Use containers for portability.
- Automate infrastructure and deployments.
- Avoid unnecessary dependence on one provider.
- Use APIs and standard interfaces.
- Support multiple environments (dev, test, prod).

Flexibility makes systems easier to update and expand.

Designing for Cost-Efficiency

Cloud costs can increase quickly without proper management.

- **Use auto-scaling.**
- **Shut down unused resources.**
- **Select the correct instance size.**
- **Use reserved or discounted pricing where suitable.**
- **Monitor resource usage.**
- **Store data using appropriate storage tiers.**
- **Remove unused disks, IPs, and snapshots.**
- **Set budgets and spending alerts.**
- **Use serverless services for suitable workloads.**

The goal is not simply to minimize cost, but to achieve the best value for the required performance and reliability.

Monitoring and Observability

Monitoring helps organizations understand system behavior.

Important Areas

- CPU and memory usage
- Application response time
- Error rates
- Network traffic
- Number of active users
- Deployment success or failure
- Cloud spending

Observability Uses

- Metrics
- Logs
- Traces
- Alerts
- Dashboards

Effective monitoring allows teams to identify problems before users are seriously affected.

Security in Cloud Deployment

Cloud management must include security from the beginning.

- **Use strong identity and access management.**
- **Apply the principle of least privilege.**
- **Encrypt data in transit and at rest.**
- **Secure container images.**
- **Regularly update software.**
- **Store secrets securely.**
- **Use network segmentation.**
- **Monitor suspicious activity.**
- **Perform vulnerability scanning.**

Security is a shared responsibility between the cloud provider and the customer.

Example: Online Shopping Application

An online shopping platform may use:

- **Containers for application services**
- **Kubernetes to manage containers**
- **Load balancer for customer requests**
- **Auto-scaling during sales events**
- **Managed database for orders**
- **Object storage for product images**

DURING A MAJOR SALE

More customers ▶ **More application Pods** ▶ **Load balancer distributes traffic** ▶ **Application remains available**

Advantages of Cloud-Based Deployment

- **Faster time to market**
- **Flexible resource usage**
- **Global application availability**
- **Automated scaling**
- **Easier disaster recovery**
- **Reduced physical infrastructure**
- **Support for CI / CD**

Successful cloud adoption requires planning, security, monitoring, and cost management.

Challenges of Cloud Management

- **Unexpected costs**
- **Security misconfigurations**
- **Vendor dependence**
- **Complex Kubernetes management**
- **Network failures**
- **Data privacy concerns**
- **Difficult troubleshooting**
- **Lack of skilled personnel**
- **Poorly designed applications**

Cloud technology provides powerful capabilities, but it does not automatically guarantee a reliable or efficient system.

Best Practices

A well-designed cloud deployment should:

- Automate infrastructure and software deployment.
- Use load balancing for critical applications.
- Configure auto-scaling carefully.
- Deploy across multiple failure zones.
- Monitor performance and cost.
- Secure identities, networks, and data.
- Test backup and recovery procedures.
- Use containers consistently.
- Keep configurations under version control.
- Review architecture regularly.

Summary

- Cloud management controls cloud infrastructure, applications, security, and costs.
- Resource provisioning makes computing resources available when needed.
- Load balancing distributes traffic across multiple servers.
- Auto-scaling adjusts capacity based on demand.
- Kubernetes orchestrates containerized applications.
- Resilient systems continue operating during failures.
- Flexible systems adapt to change.
- Cost-efficient systems use resources carefully and monitor spending.
- Security and observability are essential parts of cloud deployment.

Discussion Questions

1. What might happen if an application has no load balancer?
2. Why is auto-scaling useful during unexpected traffic increases?
3. What problem does Kubernetes solve?
4. What is the difference between horizontal and vertical scaling?
5. How can an organization reduce unnecessary cloud costs?
6. Why should cloud applications be designed to expect failures?